

Set

In []: *# Create a Set:*

```
thisset = {"apple", "banana", "cherry"}  
print(thisset)
```

In []: *# Duplicates Not Allowed
Sets cannot have two items with the same value.*

```
# Duplicate values will be ignored  
  
thisset = {"apple", "banana", "cherry", "apple"}  
  
print(thisset)
```

In []: *# The values True and 1 are considered the same value in sets, and are treated*

```
# True and 1 is considered the same value:  
  
thisset = {"apple", "banana", "cherry", True, 1, 2}  
  
print(thisset)
```

In []: *# The values False and 0 are considered the same value in sets, and are treated*

```
# False and 0 is considered the same value:  
  
thisset = {"apple", "banana", "cherry", False, True, 0}  
  
print(thisset)
```

In []: *# Get the Length of a Set
To determine how many items a set has, use the len() function.*

```
# Get the number of items in a set  
  
thisset = {"apple", "banana", "cherry"}  
  
print(len(thisset))
```

In []: *# The data type of a set*

```
myset = {"apple", "banana", "cherry"}  
print(type(myset))
```

```
In [ ]: # The set() Constructor  
# It is possible to use the set() constructor to make a set.  
  
# Use the set() constructor to make a set:  
  
thisset = set(("apple", "banana", "cherry"))  
print(thisset)
```

Access Set Items

```
In [ ]: # Access Items  
  
# Loop through the set, and print the values:  
  
thisset = {"apple", "banana", "cherry"}  
  
for x in thisset:  
    print(x)
```

```
In [ ]: # Check if "banana" is present in the set:  
  
thisset = {"apple", "banana", "cherry"}  
  
print("banana" in thisset)  
  
#Check if "banana" is NOT present in the set:  
  
thisset = {"apple", "banana", "cherry"}  
  
print("banana" not in thisset)
```

Add Set Items

```
In [ ]: # Add Items  
  
# To add one item to a set use the add() method.  
  
# Add an item to a set, using the add() method:  
  
thisset = {"apple", "banana", "cherry"}  
  
thisset.add("orange")  
  
print(thisset)
```

```
In [ ]: # Add Sets
# To add items from another set into the current set, use the update() method.

# Add elements from tropical into thisset:

thisset = {"apple", "banana", "cherry"}
tropical = {"pineapple", "mango", "papaya"}

thisset.update(tropical)

print(thisset)
```

```
In [ ]: # Add Any Iterable
#The object in the update() method does not have to be a set, it can be any it

# Add elements of a List to at set:

thisset = {"apple", "banana", "cherry"}
mylist = ["kiwi", "orange"]

thisset.update(mylist)

print(thisset)
```

Remove set Items

```
In [ ]: # Remove Item

# To remove an item in a set, use the remove(), or the discard() method.

# Remove "banana" by using the remove() method:

thisset = {"apple", "banana", "cherry"}

thisset.remove("banana")

print(thisset)
```

```
In [ ]: # Remove "banana" by using the discard() method:

thisset = {"apple", "banana", "cherry"}

thisset.discard("banana")

print(thisset)
```

```
In [ ]: # You can also use the pop() method to remove an item, but this method will re
        # The return value of the pop() method is the removed item.

        # Remove a random item by using the pop() method:

        thisset = {"apple", "banana", "cherry"}

        x = thisset.pop()

        print(x)

        print(thisset)
```

```
In [ ]: # The clear() method empties the set:

        thisset = {"apple", "banana", "cherry"}

        thisset.clear()

        print(thisset)
```

```
In [ ]: # The del keyword will delete the set completely:

        thisset = {"apple", "banana", "cherry"}

        del thisset

        print(thisset)
```

Loop Sets

```
In [ ]: # Loop Items
        # You can loop through the set items by using a for loop:

        # Loop through the set, and print the values

        thisset = {"apple", "banana", "cherry"}

        for x in thisset:
            print(x)
```

Join Sets

There are several ways to join two or more sets in Python.

The union() and update() methods joins all items from both sets.

The intersection() method keeps ONLY the duplicates.

The difference() method keeps the items from the first set that are not in the other set(s).

The symmetric difference() method keeps all items EXCEPT the duplicates.

Union

```
In [ ]: # The union() method returns a new set with all items from both sets.
```

```
# Join set1 and set2 into a new set:
```

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set3 = set1.union(set2)
```

```
print(set3)
```

```
In [ ]: # You can use the | operator instead of the union() method, and you will get the same result.
#Use | to join two sets:
```

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set3 = set1 | set2
```

```
print(set3)
```

Join Multiple Sets

```
In [ ]: # All the joining methods and operators can be used to join multiple sets.
```

```
# When using a method, just add more sets in the parentheses, separated by commas.
```

```
# Join multiple sets with the union() method:
```

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set3 = {"John", "Elena"}
```

```
set4 = {"apple", "bananas", "cherry"}
```

```
myset = set1.union(set2, set3, set4)
```

```
print(myset)
```

```
In [ ]: # When using the | operator, separate the sets with more | operators:
```

```
# Use | to join two sets:
```

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}
set3 = {"John", "Elena"}
set4 = {"apple", "bananas", "cherry"}
```

```
myset = set1 | set2 | set3 | set4
print(myset)
```

Join a Set and a Tuple

```
In [ ]:
```

```
# The union() method allows you to join a set with other data types, like List
```

```
#Join a set with a tuple:
```

```
x = {"a", "b", "c"}
y = (1, 2, 3)
```

```
z = x.union(y)
print(z)
```

Note: The | operator only allows you to join sets with sets, and not with other data types like you can with the union() method.

Update

The update() method inserts all items from one set into another.

The update() changes the original set, and does not return a new set.

```
In [ ]:
```

```
# The update() method inserts the items in set2 into set1:
```

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}
```

```
set1.update(set2)
print(set1)
```

Note: Both union() and update() will exclude any duplicate items.

Intersection

The `intersection()` method will return a new set, that only contains the items that are present in both sets.

In []:

```
# Join set1 and set2, but keep only the duplicates:

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1.intersection(set2)
print(set3)
```

You can use the `&` operator instead of the `intersection()` method, and you will get the same result.

In []:

```
# Use & to join two sets:

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1 & set2
print(set3)
```

Note: The `&` operator only allows you to join sets with sets, and not with other data types like you can with the `intersection()` method.

The `intersection_update()` method will also keep **ONLY** the duplicates, but it will change the original set instead of returning a new set.

In []:

```
# Keep the items that exist in both set1, and set2:

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set1.intersection_update(set2)

print(set1)
```

The values `True` and `1` are considered the same value. The same goes for `False` and `0`.

```
In [ ]: # Join sets that contains the values True, False, 1, and 0, and see what is co

set1 = {"apple", 1, "banana", 0, "cherry"}
set2 = {False, "google", 1, "apple", 2, True}

set3 = set1.intersection(set2)

print(set3)
```

Difference

The `difference()` method will return a new set that will contain only the items from the first set that are not present in the other set.

```
In [ ]: # Keep all items from set1 that are not in set2:

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1.difference(set2)

print(set3)
```

You can use the `-` operator instead of the `difference()` method, and you will get the same result.

```
In [ ]: # Use - to join two sets:

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1 - set2
print(set3)
```

Note: The `-` operator only allows you to join sets with sets, and not with other data types like you can with the `difference()` method.

The `difference_update()` method will also keep the items from the first set that are not in the other set, but it will change the original set instead of returning a new set.

```
In [ ]: # Use the difference_update() method to keep the items that are not present in

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set1.difference_update(set2)

print(set1)
```

Symmetric Differences

The `symmetric_difference()` method will keep only the elements that are NOT present in both sets.

```
In [ ]: # Keep the items that are not present in both sets:

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1.symmetric_difference(set2)

print(set3)
```

You can use the `^` operator instead of the `symmetric_difference()` method, and you will get the same result.

```
In [ ]: # Use ^ to join two sets:

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1 ^ set2
print(set3)
```

Note: The `^` operator only allows you to join sets with sets, and not with other data types like you can with the `symmetric_difference()` method.

The `symmetric_difference_update()` method will also keep all but the duplicates, but it will change the original set instead of returning a new set.

```
In [ ]: # Use the symmetric_difference_update() method to keep the items that are not

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set1.symmetric_difference_update(set2)

print(set1)
```

Python frozenset

frozenset is an immutable version of a set.

Like sets, it contains unique, unordered, unchangeable elements.

Unlike sets, elements cannot be added or removed from a frozenset.

Creating a frozenset

```
In [ ]: # Use the frozenset() constructor to create a frozenset from any iterable.

# Create a frozenset and check its type:

x = frozenset({"apple", "banana", "cherry"})
print(x)
print(type(x))
```

Frozenset Methods

Method	Shortcut	Description
<code>copy()</code>		Returns a shallow copy
<code>difference()</code>	-	Returns a new frozenset with the difference
<code>intersection()</code>	&	Returns a new frozenset with the intersection
<code>isdisjoint()</code>		Returns whether two frozensets have an intersection
<code>issubset()</code>	<code><=</code> / <code><</code>	Returns True if this frozenset is a (proper) subset of another
<code>issuperset()</code>	<code>>=</code> / <code>></code>	Returns True if this frozenset is a (proper) superset of another
<code>symmetric_difference()</code>	^	Returns a new frozenset with the symmetric difference
<code>union()</code>	,	,

Set Methods

Python has a set of built-in methods that you can use on sets.

Method	Shortcut	Description
<code>add()</code>		Adds an element to the set
<code>clear()</code>		Removes all the elements from the set
<code>copy()</code>		Returns a copy of the set
<code>difference()</code>	-	Returns a set containing the difference between two or more sets

Method	Shortcut	Description
difference_update()	-=	Removes the items in this set that are also included in another, specified set
discard()		Remove the specified item (no error if not found)
intersection()	&	Returns a set, that is the intersection of two other sets
intersection_update()	&=	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()		Returns whether two sets have an intersection or not
issubset()	<=	Returns True if all items of this set is present in another set
	<	Returns True if all items of this set is present in another, larger set
issuperset()	>=	Returns True if all items of another set is present in this set
	>	Returns True if all items of another, smaller set is present in this set
pop()		Removes an element from the set (randomly)
remove()		Removes the specified element (raises error if not found)
symmetric_difference()	^	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	^=	Inserts the symmetric differences from this set and another

In []: